Artifical Intelligence Basics Course Summary

Generated by Scribe.AI

December 9, 2024

1 Key Terms

- **Minimax Search**: A search algorithm for two-player zero-sum games that aims to find the best move for the current player, assuming the opponent also plays optimally.
- **Alpha-Beta Pruning**: A technique to reduce the search space in Minimax search by avoiding the evaluation of branches that cannot affect the optimal decision.
- **Turing Test**: An operational test for intelligent behavior where a human interrogator tries to distinguish between a human and a machine based on their text-based responses.
- **Game Tree**: A tree-like structure representing all possible game states and moves, starting from the initial state and branching out to all possible outcomes.
- **Perceptron**: A fundamental unit in neural networks that takes multiple inputs, applies weights to them, sums the weighted inputs, and produces an output based on a threshold.
- **Reinforcement Learning**: A type of machine learning where an agent learns to make decisions by interacting with an environment and receiving rewards or penalties.
- **Matrix**: A rectangular array of numbers, symbols, or expressions, arranged in rows and columns.
- **Linear Transform**: A function that maps vectors from one vector space to another, satisfying $T(u+v) = T(u) + T(v)$ and $T(cu) = cT(u)$ for all vectors u, v and scalars c.
- **Matrix Multiplication**: An operation that combines two matrices to produce a third matrix, where the element at row i and column j of the resulting matrix is the dot product of row i of the first matrix and column j of the second matrix. The inner dimensions must match.
- **Matrix Inverse**: For a square matrix A, its inverse A^{-1} is a matrix such that $AA^{-1} = A^{-1}A = I$, where I is the identity matrix.
- **Matrix Transposition**: An operation that swaps the rows and columns of a matrix. The transpose of matrix A is denoted as A^T.
- **Derivative**: A measure of how a function changes in response to a small change in its input. It represents the instantaneous rate of change.
- **Partial Derivative**: A derivative of a function of multiple variables with respect to one of those variables, while holding the others constant.
- **Sigmoid Function**: A mathematical function that maps any input value to an output value between 0 and 1. Often used as an activation function in neural networks.
- **Inductive Reasoning**: A type of reasoning that involves drawing general conclusions from specific observations.

- **Machine Learning (ML)**: The process of using algorithms to parse data, learn from it, and then make a determination or prediction about something.
- **Linear Regression**: A statistical method used to model the relationship between a dependent variable and one or more independent variables by fitting a linear equation to observed data.
- **Overfitting**: A phenomenon in machine learning where a model learns the training data too well, resulting in poor generalization to new, unseen data.
- **Binary Classification**: A type of classification problem where the goal is to assign data points to one of two categories.
- **Decision Boundary**: A line or surface that separates data points into different classes in a classification problem.
- **0-1 Loss**: A loss function in classification that assigns a loss of 1 if the prediction is incorrect and 0 if it is correct.
- **Perceptron Loss**: A loss function used in the perceptron algorithm that penalizes misclassifications.
- **Hinge Loss**: A loss function used in support vector machines (SVMs) that penalizes predictions that are not sufficiently confident.
- **Logistic Loss**: A loss function used in logistic regression that measures the difference between the predicted probability and the true label.
- **Gradient Descent**: An iterative optimization algorithm used to find the minimum of a function by following the negative gradient.
- **Stochastic Gradient Descent**: A variant of gradient descent that updates the model parameters using a randomly selected subset of the data.
- **Perceptron Algorithm**: An iterative algorithm used for binary classification that updates the model parameters based on misclassified data points.
- **Bag of Features**: A representation of an image as a collection of visual features (visual words), disregarding their spatial arrangement.
- **Semantic Gap**: The difference between how humans perceive images (semantically) and how computers represent them (numerically).
- **Visual Words**: Basic visual elements extracted from images, used to represent image content in a way that is less sensitive to variations in viewpoint, illumination, etc.
- **Invariance**: The property of extracted features to remain consistent across different variations of an object (e.g., viewpoint changes, lighting conditions).
- **Robustness**: The ability of extracted features to withstand noise, detection errors, illumination changes, partial occlusion, and shape distortions.
- **Textons**: Basic elements or patterns that characterize a texture.
- **Mean Filtering**: A noise reduction technique that replaces each pixel with the average of its neighboring pixels.
- **Weighted Moving Average**: A noise reduction technique similar to mean filtering, but assigns different weights to neighboring pixels.
- **Image Filtering**: A process of computing a function of the local neighborhood at each pixel in an image, often using a filter or mask.

- **Identity Filter**: An image filter that leaves the image unchanged.
- **Sharpening**: An image enhancement technique that increases the contrast between adjacent pixels, making edges and details more pronounced.
- **Nearest Neighbor Classifier**: A classification algorithm that assigns the label of the nearest training data point to each test data point.
- **Category Models**: Representations of different object categories in a model space, often based on feature vectors.
- **Codewords**: Visual features extracted from an image, often represented as small image patches.
- **Edge Detection**: A technique in image processing used to identify points of significant intensity change in an image.
- **Edge Strength**: The magnitude of the image gradient, indicating the strength of an edge.
- **Feature Vectors**: Numerical representations of features extracted from an image or other data, used for classification or other machine learning tasks.
- **Intensity Function**: A function that maps each pixel in an image to its intensity value.
- **Intensity Profile**: A plot of intensity values along a single scanline of an image.
- **Model Space**: A multi-dimensional space where each point represents a category model.
- **Perceptual Organization**: The process of grouping pixels into meaningful objects or regions in an image.
- **Vision Pyramid**: A hierarchical representation of an image at multiple scales, often created by repeatedly blurring and subsampling the image.
- **Convolutional Neural Network (CNN)**: A type of artificial neural network used for processing data that has a known grid-like topology, such as an image. CNNs use convolutional layers to extract features from the input data.
- **ImageNet**: A large dataset of images used for training and evaluating image recognition models. It contains millions of images across thousands of categories.
- **AlexNet**: One of the first successful deep convolutional neural networks, known for its use of GPUs and its winning performance in the ImageNet challenge.
- **Dense Neural Network**: A type of neural network where each neuron in one layer is connected to every neuron in the next layer.
- **ConvNet**: Short for Convolutional Neural Network.
- **Artificial Neuron**: A mathematical model of a biological neuron, typically consisting of weighted inputs, a summation function, and an activation function.
- **Convolution Layer**: A layer in a convolutional neural network that applies a filter (kernel) to the input data to extract features.
- **Pooling Layer**: A layer in a convolutional neural network that reduces the spatial dimensions of the feature maps, typically using max pooling or average pooling.
- **Local Receptive Field**: The small region of the input data that a neuron in a convolutional layer is connected to.
- **Sparse Connectivity**: A characteristic of convolutional neural networks where each neuron is connected to only a few neurons in the previous layer, unlike fully connected networks.

- **Parameter Sharing**: A technique used in convolutional neural networks where the same parameters (weights) are used across different spatial locations in the input data.
- **Rectified Linear Unit (ReLU)**: A non-linear activation function defined as $f(x) = \max(0, x)$.
- **Max Pooling**: A pooling operation that selects the maximum value within a defined region of the input data.
- **Average Pooling**: A pooling operation that calculates the average value within a defined region of the input data.
- **Back-propagation**: An algorithm used to train neural networks by calculating the gradient of the loss function with respect to the network's weights and updating the weights accordingly.
- **Fully Connected Layer (FC)**: A layer in a neural network where each neuron is connected to every neuron in the previous layer.
- **MNIST**: A large database of handwritten digits used for training and testing machine learning models.
- **LeNet-5**: A convolutional neural network architecture specifically designed for the MNIST dataset.
- **CIFAR-10**: A large dataset of 32x32 color images, commonly used for training and evaluating image classification models.
- **Held Out Data**: A subset of the training data used to tune hyperparameters of a machine learning model.
- **Test Data**: A subset of the data used to evaluate the performance of a trained machine learning model on unseen data.
- **Training Data**: The subset of data used to train a machine learning model.
- **Regularization Coefficient**: A hyperparameter (λ) in regularized regression that controls the trade-off between training error and model complexity.
- **Regularizer**: A penalty term added to the loss function in regularized regression to prevent overfitting by constraining the magnitude of the model's weights.
- **LASSO**: A type of regularization that uses the L1 norm of the weights as the regularizer.
- **Learning Curves**: Graphs that show the relationship between model complexity and accuracy on both training and test sets, used to diagnose underfitting and overfitting.
- **S-fold Cross Validation**: A model evaluation technique where the data is split into S parts, and each part is used as a validation set once while the others are used for training.
- **Data Analysis Pipeline**: A sequence of steps involved in analyzing data, including data collection, model design, parameter inference, and model application.
- **Matrix Factorization**: A technique that decomposes a large matrix into the product of two smaller matrices, revealing latent factors that capture underlying relationships in the data.
- **Principal Component Analysis (PCA)**: A linear dimensionality reduction technique that identifies the principal components (directions of maximum variance) in a dataset to reduce dimensionality while preserving important information.
- **Dimensionality Reduction**: Techniques that transform data from a higher-dimensional space to a lower-dimensional space, reducing the number of attributes while retaining essential information.
- **Low Rank Matrix Approximations**: Approximating a high-dimensional matrix with a lower-rank matrix, effectively reducing the dimensionality of the data by representing it with fewer latent factors.

- **Embeddings**: A low-dimensional representation of data points, often obtained through dimensionality reduction techniques like matrix factorization, that captures the essential relationships between data points.
- **Eigenvectors**: Vectors that, when multiplied by a matrix, result in a scalar multiple of themselves (eigenvalues). In PCA, eigenvectors represent the principal components.
- **Eigenvalues**: Scalars that represent the scaling factor by which eigenvectors are multiplied when transformed by a matrix. In PCA, eigenvalues represent the variance explained by each principal component.
- **Collaborative Filtering**: A recommender system technique that predicts user preferences based on the preferences of similar users.
- **Recommender Systems**: Systems that predict user preferences for items (e.g., movies, products) based on past user behavior and other relevant data.
- **Root Mean Square Error (RMSE)**: A metric used to evaluate the accuracy of a recommender system by calculating the square root of the average squared differences between predicted and actual ratings.
- **Bias-Variance Tradeoff**: The balance between model complexity (variance) and the accuracy of the model on the training data (bias). A model with high variance may overfit the training data, while a model with high bias may underfit the data.
- **Single Nucleotide Polymorphisms (SNPs)**: Variations in a single nucleotide base (A, C, G, or T) in a DNA sequence, commonly used in genetic studies.
- **Clustering**: The process of grouping a set of objects into classes of similar objects. Objects within a cluster should be similar, and objects from different clusters should be dissimilar.
- **K-means clustering**: A commonly used clustering algorithm that partitions objects into K disjoint subsets, assuming the objects are p-dimensional vectors and using a distance measure (typically Euclidean distance) between them.
- **Euclidean distance**: The most commonly used distance measure for continuous data, defined as $d_E(x, y) = \sqrt{\sum_{i=1}^{p}(x_i - y_i)^2}$, where p is the number of attributes/dimensions.
- **Hamming distance**: A distance measure used for bit/binary vectors that counts the number of bits that differ. It can be generalized to discrete data to count mismatches.
- **Matching coefficient**: A normalized Hamming distance, obtained by dividing the Hamming distance by the number of attributes/bits.
- **Inertia**: In the context of K-means, the sum of squared distances from each point to its cluster centroid. Minimizing inertia is the goal of K-means optimization.
- **Purity**: A measure of the extent to which clusters contain objects of a particular class. It's calculated as $\text{purity}(C, G) = \frac{1}{N}\sum_{i=1}^{K}\max_j N_j \in G_i$, where N is the total number of points, K is the number of clusters, and N_j is the number of examples of class j in cluster i.
- **Personas**: Representations of different user types or target audiences, used to provide a common understanding across teams.
- **Propositional logic**: A formal system of logic that uses propositions (statements that can be true or false) and logical connectives to express complex logical relationships.
- **Model**: In propositional logic, an assignment of truth values (true or false) to each propositional variable.

- **Logical Equivalence**: A relationship between two logical expressions that have the same truth value under all possible assignments of truth values to their variables.
- **Validity**: A property of a logical sentence that is true in all possible models (interpretations).
- **Satisfiability**: A property of a logical sentence that is true in at least one model (interpretation).
- **Unsatisfiability**: A property of a logical sentence that is false in all possible models (interpretations).
- **Conjunctive Normal Form (CNF)**: A standardized way of representing logical expressions as a conjunction of clauses, where each clause is a disjunction of literals.
- **Literal**: In propositional logic, a literal is a propositional variable or its negation.
- **Clause**: In propositional logic, a clause is a disjunction of literals.
- **Satisfying Assignment**: An assignment of truth values to variables that makes a logical formula true.
- **Boolean Satisfiability Problem (SAT)**: The computational problem of determining whether a given Boolean formula is satisfiable.
- **3-Satisfiability (3-SAT)**: A restricted version of the SAT problem where each clause in the Boolean formula contains exactly three literals.
- **Equisatisfiable**: Two logical formulas are equisatisfiable if they have the same set of satisfying assignments.
- **Intelligent Agent**: A system that perceives its environment and takes actions that maximize its chances of success.
- **Automated Reasoning**: The process of using computers to solve problems that require logical reasoning.
- **Model Checking**: A technique for verifying the correctness of hardware or software systems by checking if a model of the system satisfies a given property.
- **Bounded Model Checking (BMC)**: A model checking technique that explores a finite number of steps in the system's execution.
- **Classical Planning**: A type of planning problem where the world is deterministic and fully observable.
- **Combinatorial Design**: The study of mathematical structures with specific properties, often used in various applications like experimental design and cryptography.
- **Recursion**: A process where a function calls itself, eventually terminating in a base case.
- **Base Case**: The simplest form of a problem in a recursive approach, providing a direct solution without further recursion.
- **Recursive Case**: In a recursive approach, the same problem/method/data structure, but smaller, closer to the base case.
- **Breadth First**: A tree traversal method that visits nodes level by level.
- **Depth First**: A tree traversal method that traverses down a branch of a tree.
- **Binary Search Tree**: A tree data structure where each node has a value, its left child has a smaller value, and its right child has a larger value.
- **Problem Solving as Search**: An approach to problem-solving that involves exploring possibilities to find a solution.
- **States**: Configurations of an environment in a search problem.

- **Actions**: Activities that move from one state to another in a search problem.
- **Search Algorithm**: Determines which actions should be tried in which order to find a solution in a search problem.
- **Redundant States**: States that are visited multiple times during a search, leading to inefficiency.
- **Dynamic Programming**: An optimization technique that avoids redundant calculations by storing and reusing previously computed values.
- **Graph**: A data structure consisting of nodes and edges, representing relationships between elements.
- **World State**: Every detail of the environment in a search problem.
- **Search State**: Only the details needed for planning in a search problem.
- **Rational Agent**: An agent that acts to maximize its goal achievement given available information.
- **Davis-Putnam-Logemann-Loveland (DPLL) Algorithm**: A recursive algorithm for solving the SAT problem by systematically exploring possible variable assignments.
- **Hamiltonian Path Problem**: The problem of determining if there exists a path in a graph that visits each vertex exactly once.
- **Graph Coloring**: The problem of assigning colors to the vertices of a graph such that no two adjacent vertices have the same color.
- **Data Science**: The interdisciplinary field that uses scientific methods, processes, algorithms and systems to extract knowledge and insights from structured and unstructured data.
- **Bias and Fairness in Data**: The presence of systematic and repeatable errors in data that can lead to unfair or discriminatory outcomes.
- **Automatic Prediction System**: A system that uses algorithms and data to make predictions about future events or outcomes.
- **Anonymized Data**: Data that has had identifying information removed to protect individual privacy.
- **Privacy**: The state or condition of being free from being observed or disturbed by other people.
- **Security**: The state of being free from danger or threat.
- **Disparate Treatment**: A type of discrimination where individuals are treated differently based on their membership in a protected group.
- **Disparate Impact**: A type of discrimination where a seemingly neutral policy or practice has a disproportionately negative effect on a protected group.
- **Proxies for Discrimination**: Variables that are not directly discriminatory but are correlated with protected characteristics and can lead to discriminatory outcomes.
- **Interpretable Results**: Results from a machine learning model that are easily understandable and explainable.

2 Problem Types

- **Understanding the Nature of Intelligence**: This problem explores the definition and characteristics of intelligence, both human and artificial. Different perspectives and definitions of AI are examined.
- **Evaluating Intelligent Behavior**: This problem focuses on developing methods for assessing whether a machine exhibits intelligent behavior. The Turing Test and its limitations are discussed, along with counterarguments like Searle's Chinese Room argument.
- **Overcoming the Limits of Search**: This problem addresses the computational challenges of exhaustive search in complex games like chess and Go. The exponential growth of search space is highlighted, leading to the need for machine learning techniques to supplement search.
- **Addressing AI Safety and Ethical Concerns**: This problem explores the potential risks and ethical implications of AI systems. Examples include bias in algorithms and the vulnerability of neural networks to adversarial attacks.
- **Bridging the Gap Between Different AI Domains**: This problem investigates the commonalities and differences between seemingly disparate AI tasks, such as game playing and speech synthesis. The shared algorithmic components and challenges are analyzed.
- **Understanding the AI Stack and its Components**: This problem examines the different layers and components that constitute a complete AI system, such as machine learning, search algorithms, computer vision, and human-computer interaction.
- **Analyzing the Architecture of Successful AI Systems**: This problem focuses on understanding the design and functionality of successful AI systems, such as AlphaGo, by examining their components and how they interact.
- **Representing Linear Transformations with Matrices**: Matrices are used to represent linear transformations, which map vectors from one space to another while preserving linear combinations.
- **Performing Matrix Arithmetic**: This involves addition, subtraction, scalar multiplication, and multiplication of matrices, with specific rules and constraints on dimensions.
- **Solving Systems of Linear Equations**: Systems of linear equations can be represented and solved using matrices and their inverses.
- **Understanding Perceptrons**: A perceptron is a fundamental building block of neural networks, represented as a weighted sum of inputs followed by an activation function.
- **Parallelizing Matrix Multiplication**: Matrix multiplication can be parallelized using techniques like domain decomposition to improve computational efficiency.
- **Calculating Derivatives**: Derivatives, including partial derivatives, are essential for optimization algorithms used in AI, such as gradient descent.
- **Defining Machine Learning**: Machine learning is defined as automated inductive reasoning, illustrated by examples of classifying trains carrying toxic chemicals and those that do not.
- **Image Recognition Challenges**: The difficulty of teaching computers to perform image recognition is highlighted using examples of sheep, horses, and an alpaca, posing the question of how to differentiate between these animals based on visual data.
- **Building Machine Learning Models**: The process of building a model from past examples is explained, emphasizing the importance of using representative data to accurately reflect the problem space.
- **Linear Regression as the First ML Problem**: Linear regression is introduced as the first machine learning problem, using the example of predicting house prices based on square footage. The challenge of finding the "best" line and defining "best" is discussed.

- **Multiple Linear Regression**: Extending linear regression to include multiple independent variables is discussed, using the example of predicting CO2 emissions from car weight and volume.
- **Curve Fitting and Overfitting**: The concept of fitting curves to data and the potential for overfitting are illustrated using polynomial regression of varying degrees.
- **Binary Classification**: Binary classification is introduced as a problem of learning a mapping from a feature vector to a label (+1 or -1), illustrated with examples of classifying animals and a scatter plot with a decision boundary.
- **Choosing Loss Functions for Binary Classification**: The challenges of using the 0-1 loss function are discussed, and alternative loss functions (perceptron, hinge, and logistic loss) are introduced.
- **Minimizing Loss Functions**: The process of minimizing loss functions using gradient descent is explained, with a focus on stochastic gradient descent.
- **Perceptron Algorithm**: The Perceptron algorithm is presented as a method for finding a separating hyperplane in linearly separable data.
- **Logistic Regression**: Logistic regression is introduced as a method for binary classification using the logistic loss function and the sigmoid function.
- **Image Classification Challenges**: This problem focuses on the difficulties in accurately classifying images due to variations in viewpoint, illumination, deformation, occlusion, and background clutter.
- **Representing Images Numerically**: This problem addresses the challenge of representing images as numerical data that can be processed by machine learning algorithms, highlighting the semantic gap between human perception and computer representation.
- **Choosing an Appropriate Model for Image Classification**: This problem explores the suitability of different machine learning models (e.g., logistic regression) for image classification and their limitations.
- **Finding Invariant Features**: This problem focuses on identifying features in images that remain consistent despite variations in viewpoint, lighting, and other factors, crucial for robust image classification.
- **Bag of Features Approach**: This problem explores the Bag of Features approach to image classification, which represents images as collections of visual words and their frequencies.
- **Noise Reduction in Images**: This problem focuses on techniques for reducing noise in images, including mean filtering and moving average filters, to improve image quality and analysis.
- **Image Sharpening**: This problem explores techniques for enhancing image sharpness by reversing the effects of blurring.
- **Document Representation for Retrieval**: This problem addresses the challenge of representing documents numerically for efficient retrieval, using word frequencies as features.
- **Bag of Features Object Categorization**: This problem focuses on using the Bag of Features approach to categorize objects based on their extracted features, represented as codewords and their frequencies. The categorization is achieved by mapping feature vectors into a model space.
- **Edge Detection**: This problem involves identifying edges in images, which are curves where brightness changes significantly. Edges are useful for object recognition, understanding 3D structure, and grouping pixels into objects.
- **Corner Detection**: This problem aims to identify corners in images, which are points where the image intensity changes significantly in multiple directions. The approach involves analyzing the changes in pixel intensities when a small window is translated across the image.

- **Object Detection and Recognition using CNNs**: Convolutional Neural Networks are used to identify and locate objects within images and videos. Examples include facial feature detection, car detection in highway scenes, and fruit recognition.
- **Improving ImageNet Classification Accuracy**: This problem focuses on improving the accuracy of image classification models over time, as demonstrated by the decreasing error rate in the ImageNet challenge. AlexNet is highlighted as a significant milestone.
- **Comparing Dense and Convolutional Neural Networks**: This problem involves understanding the architectural differences between standard fully connected neural networks and convolutional neural networks, particularly in terms of neuron arrangement and data processing in multiple dimensions.
- **Understanding Artificial Neurons**: This problem focuses on understanding the fundamental operations of an artificial neuron, including weighted input summation and the application of activation functions to introduce non-linearity.
- **Convolution and Pooling Operations in CNNs**: This problem involves understanding the mathematical operations of convolution and pooling layers in CNNs, including the application of convolution filters and pooling functions to reduce spatial dimensions.
- **Explaining Sparse Connectivity in Convolutional Layers**: This problem explores the reasons behind using convolutional layers instead of fully connected layers, focusing on the concept of sparse connectivity and its advantages in terms of computational efficiency and parameter sharing.
- **Understanding the ReLU Activation Function**: This problem involves understanding the mathematical definition and graphical representation of the Rectified Linear Unit (ReLU) activation function.
- **Understanding Pooling Layers**: This problem focuses on understanding the function of pooling layers in CNNs, including max pooling and average pooling, and their role in downsampling the input volume.
- **Backpropagation for Neural Network Training**: This problem involves understanding the backpropagation algorithm used to train neural networks by adjusting weights to minimize the error between predicted and desired outputs.
- **Understanding a Simple CNN Structure**: This problem involves understanding the architecture of a simple CNN, including the sequence of convolutional, activation, pooling, and fully connected layers.
- **Analyzing Learned Convolution Filters**: This problem involves analyzing the patterns and features learned by convolution filters in a CNN, as visualized in a grid of learned filters.
- **Multidimensional Convolution**: This problem involves understanding how multidimensional convolution operates on images with multiple color channels (e.g., RGB).
- **Biological Inspiration for Artificial Neural Networks**: This problem involves understanding the biological inspiration behind artificial neural networks, drawing parallels between biological neurons and perceptrons.
- **Using the MNIST Dataset for Handwritten Digit Recognition**: This problem involves understanding the MNIST dataset and its use in training and testing machine learning models for handwritten digit recognition.
- **LeNet-5 Architecture for MNIST**: This problem involves understanding the architecture of LeNet-5, a CNN specifically designed for the MNIST dataset, including the sequence and parameters of its layers.
- **CIFAR-10 Dataset and State-of-the-Art Models**: This problem involves understanding the CIFAR-10 dataset and comparing the performance of different state-of-the-art deep learning models on this dataset.
- **Generalization and Overfitting**: This problem focuses on the ability of a machine learning model to generalize to unseen data, and the risk of overfitting the training data.

- **Training \textbackslash{}& testing errors**: This problem involves analyzing the prediction errors on training and testing datasets to understand the bias-variance tradeoff and diagnose overfitting or underfitting.
- **Training and Testing**: This problem addresses the process of splitting data into training, held-out (validation), and test sets for model development and evaluation.
- **How to Diagnose Overfitting**: This problem describes a method to identify overfitting by comparing a model's performance on training and test data.
- **Hyper-Parameter Tuning**: This problem involves finding the optimal settings for hyperparameters in a machine learning model to improve performance.
- **Fit a cubic function with a degree-9 polynomial**: This problem illustrates overfitting by fitting a high-degree polynomial to data, resulting in a complex model that doesn't generalize well.
- **Control the Model Complexity**: This problem explores methods to control model complexity, such as using regularization or limiting the number of parameters.
- **Tuning on Held-Out Validation Data**: This problem focuses on using a held-out validation set to tune hyperparameters and prevent overfitting.
- **Learning curves illuminate likely causes of error**: This problem involves interpreting learning curves (plots of accuracy vs. model complexity) to diagnose underfitting or overfitting.
- **When to get more data vs. use different model**: This problem addresses the decision of whether to gather more data or change the model based on learning curves and the presence of underfitting or overfitting.
- **S-fold cross validation**: This problem describes a technique for model evaluation and hyperparameter tuning using cross-validation.
- **The Data Analysis Pipeline**: This problem outlines the stages of a data analysis pipeline, highlighting potential issues at each stage.
- **Approximating High-Dimensional Data with Lower-Dimensional Models**: Matrix factorization techniques are used to approximate high-dimensional data using lower-dimensional representations, capturing latent factors that explain the data's structure.
- **Dimensionality Reduction using SVD and PCA**: Singular Value Decomposition (SVD) and Principal Component Analysis (PCA) are used to reduce the dimensionality of data while preserving important information. PCA finds principal components that maximize variance.
- **Recommender System Algorithm Design**: Recommender systems aim to predict user ratings for items they haven't interacted with. Collaborative filtering is a common approach, using ratings from similar users. Matrix factorization is used to infer latent factors for users and items.
- **Handling Missing Data in Recommender Systems**: Recommender systems often deal with sparse data, where many user-item ratings are missing. Strategies for handling missing data are discussed, including imputation and optimization techniques.
- **Modeling Systematic Biases in Ratings**: User and item biases can significantly affect ratings. Methods for modeling these biases to improve prediction accuracy are explored.
- **Bias-Variance Tradeoff in Recommender Systems**: The tradeoff between model complexity and prediction accuracy is analyzed. More complex models can overfit, while simpler models may underfit.
- **Clustering Data**: Grouping a set of objects into classes of similar objects, where objects within a cluster are similar and objects from different clusters are dissimilar. This is a common unsupervised learning task with applications in various fields.

- **Supervised vs. Unsupervised Learning**: Distinguishing between supervised learning (given paired inputs/outputs to learn a function f such that $f(X) = y$) and unsupervised learning (given inputs X to learn a function f to simplify the data, $f(X) = y$). Examples of each are provided.
- **Defining Clustering**: Describing clustering as the process of grouping objects into classes of similar objects, with the goal of maximizing intra-cluster similarity and minimizing inter-cluster similarity.
- **Choosing the Number of Clusters (k) in K-means**: Determining the optimal number of clusters (k) in K-means clustering, balancing accuracy (low inertia) and complexity (small k) using methods like the "elbow" method.
- **Evaluating Clustering Results**: Assessing the quality of clustering results, considering both subjective (visual inspection of proximity matrices and PCA-reduced visualizations) and objective (purity and similarity measures using class labels) approaches.
- **Measuring Similarity/Distance**: Using distance measures like Euclidean distance ($d_E(x, y) = \sqrt{\sum_{i=1}^{p}(x_i - y_i)^2}$) for continuous data and Hamming distance for discrete data (binary vectors), and the matching coefficient.
- **K-means Clustering Algorithm**: Understanding the steps involved in the K-means algorithm, including initialization, data assignment, and relocation of means.
- **Defining Reasoning Using Logic**: Reasoning is defined as proving a conclusion from given assumptions using rules of deduction. Examples include proving mathematical theorems or determining winning moves in games.
- **Defining the Satisfiability Problem (SAT)**: The SAT problem involves determining whether a given propositional formula is satisfiable, meaning there exists an assignment of truth values to its variables that makes the formula true.
- **Converting Logical Expressions to Conjunctive Normal Form (CNF)**: This problem focuses on transforming arbitrary logical expressions into an equivalent CNF representation, which is a conjunction of clauses, each being a disjunction of literals.
- **Applying SAT to Model Checking**: This problem involves using SAT solvers to verify properties of hardware or software systems by encoding the system's behavior and properties into a CNF formula and checking its satisfiability.
- **Applying SAT to Classical Planning**: This problem involves using SAT solvers to find plans that achieve a goal in a given environment by encoding the planning problem into a CNF formula and finding a satisfying assignment.
- **Applying SAT to Combinatorial Design**: This problem involves using SAT solvers to find complex mathematical structures with desirable properties, such as Latin squares or Steiner systems, by encoding the constraints of the design into a CNF formula.
- **Reducing SAT to 3-SAT**: This problem involves transforming a general SAT problem into an equivalent 3-SAT problem, where each clause contains exactly three literals. This is done by introducing new variables and creating new clauses that preserve satisfiability.
- **Representing Recursion**: This problem focuses on understanding and implementing recursive algorithms and data structures, such as the Fibonacci sequence and tree traversal.
- **Optimizing Recursive Algorithms**: This problem involves identifying and eliminating redundant calculations in recursive algorithms to improve efficiency, as demonstrated with the Fibonacci sequence.
- **Representing Problems as Search**: This problem involves formulating problems, such as mazes and the 8-puzzle, as search problems by defining states, actions, and goal tests.

- **Searching Graphs and Maps**: This problem focuses on extending search algorithms to graph-based structures, considering multiple paths to a solution and the computational complexity of searching large state spaces.
- **Defining State Spaces**: This problem involves defining appropriate state representations for different problems, distinguishing between world states and search states, and analyzing the size of the state space.
- **Designing Rational Agents**: This problem focuses on understanding the concept of rational agents and how their actions should maximize goal achievement given available information.
- **Solving the Boolean satisfiability problem (SAT)**: This problem involves determining whether there exists an assignment of truth values to variables that satisfies a given Boolean formula in conjunctive normal form (CNF).
- **General automated reasoning**: This involves developing techniques to improve reasoning and modeling technology for solving problems across various domains, such as logistics, chess, planning, and verification.
- **Addressing exponential complexity growth**: This addresses the challenge of exponential complexity growth in various problem domains, such as VLSI verification, military logistics, chess, and deep space mission control.
- **Encoding real-world problems into SAT**: This involves translating real-world problems, such as the Hamiltonian path problem and graph coloring, into SAT instances that can be solved using SAT solvers.
- **Identifying and Mitigating Bias in Data and Algorithms**: This problem focuses on identifying and addressing biases present in datasets and the resulting algorithms, which can lead to unfair or discriminatory outcomes. Examples include bias in hiring algorithms (Slide 23), predictive policing (Slide 16), and online advertising (Slide 14).
- **Ensuring Data Privacy and Security**: This problem involves protecting sensitive personal information used in data analysis and AI systems. The challenge lies in balancing the need for data with the ethical imperative to protect individual privacy. Examples include anonymization techniques and their limitations (Slide 10, Slide 11).
- **Defining and Achieving Fairness in Algorithmic Outcomes**: This problem explores different definitions of fairness (e.g., group fairness, individual fairness) and how to achieve them in AI systems. The difficulty lies in operationalizing these definitions and balancing competing fairness criteria.
- **Improving the Interpretability and Trustworthiness of AI Models**: This problem focuses on making AI models more transparent and understandable, so that their outputs can be trusted and their decisions can be explained. The challenge lies in balancing model complexity with the need for interpretability.

3 Algorithm Solutions

- **Minimax Search**: A recursive algorithm for two-player zero-sum games that explores the game tree to find the best move for the current player, assuming the opponent also plays optimally. It alternates between maximizing the score for the current player and minimizing the score for the opponent.
- **Alpha-Beta Pruning**: An optimization technique for Minimax search that reduces the search space by avoiding the evaluation of branches that cannot affect the optimal decision. It uses alpha and beta values to prune branches.
- **Matrix Addition**: $A + B = C$, where $c_{ij} = a_{ij} + b_{ij}$
- **Matrix Scalar Multiplication**: $k \cdot A = B$, where $b_{ij} = k \cdot a_{ij}$

- **Matrix Multiplication**: $C_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$
- **Matrix Transposition**: Swapping rows and columns of a matrix A to obtain A^T.
- **Solving Linear Equations with Invertible Matrices**: $Ax = b \implies x = A^{-1}b$
- **Derivative of Sigmoid Function**: $\frac{d}{dx}\sigma(x) = \frac{e^{-x}}{(1+e^{-x})^2}$
- **Multiple Linear Regression**: Find the vector β that minimizes the sum of squared errors in the model $y = X\beta + \epsilon$, where y is the vector of dependent variables, X is the matrix of independent variables, β is the vector of coefficients, and ϵ is the error vector. The solution is given by $\beta^{=}(X^T X)^{-1} X^T y$.
- **Perceptron Algorithm**: Iteratively update the weight vector w by randomly selecting a misclassified data point (x_n, y_n) and updating $w \leftarrow w + y_n x_n$.
- **Bag of Features**: Represent images by frequencies of "visual words" (Bags of features)
- **Nearest Neighbor Classifier**: Assign label of nearest training data point to each test data point
- **Moving Average**: Replace each pixel with an average of all the values in its neighborhood
- **Weighted Moving Average**: Add weights to our moving average
- **Image Sharpening**: $f_{sharp} = f + \alpha(f - f_{blur})$
- **Finite Difference**: $\frac{df}{dx}[x, y] \approx F[x+1, y] - F[x, y]$
- **Image Gradient Magnitude**: $||\nabla f|| = \sqrt{(\frac{\partial f}{\partial x})^2 + (\frac{\partial f}{\partial y})^2}$
- **Image Gradient Direction**: $\theta = \tan^{-1}(\frac{\partial f}{\partial x} / \frac{\partial f}{\partial y})$
- **Sum of Squared Differences (SSD)**: $E(u, v) = \sum_{(x,y) \in W} [I(x+u, y+v) - I(x, y)]^2$
- **Anisotropic Gaussian**: $G_{\sigma_x, \sigma_y}(x, y) = \frac{1}{2\pi\sigma_x\sigma_y} \cdot e^{-\frac{1}{2}((\frac{x^2}{\sigma_x^2}) + (\frac{y^2}{\sigma_y^2}))}$
- **Isotropic Gaussian**: $G_\sigma(x, y) = \frac{1}{2\pi\sigma^2} \cdot e^{-\frac{r^2}{2\sigma^2}}$
- **Convolution**: A convolution operation involves sliding a filter (kernel) across an input image, performing element-wise multiplication between the filter and the corresponding input region, and summing the results to produce a single output value. This process is repeated for every possible position of the filter on the input image, resulting in an output feature map.
- **ReLU Activation Function**: The ReLU (Rectified Linear Unit) activation function is defined as $f(x) = max(0, x)$. It outputs the input value if it's positive, and 0 otherwise. This introduces non-linearity into the network.
- **Max Pooling**: Max pooling is a downsampling operation that selects the maximum value within a defined region (e.g., a 2x2 window) of a feature map. This reduces the spatial dimensions of the feature map while retaining the most prominent features.
- **Backpropagation**: Backpropagation is an algorithm used to train neural networks. It calculates the gradient of the loss function with respect to the network's weights and uses this gradient to update the weights iteratively, minimizing the error between the network's output and the desired output.

- **Regularized Linear Regression**: $E(w) = RSS(w) + \lambda R(w)$, where $R(w) = ||w||_2^2 = w_1^2 + ... + w_k^2$ or $R(w) = ||w||_1 = |w_1| + ... + |w_k|$.
- **S-fold Cross Validation**: Split training data into S equal parts; use each part as validation data, others as training data; choose hyperparameter with best average performance.
- **Singular Value Decomposition (SVD)**: $A = USV^T$, where A is the input matrix, U and V are orthogonal matrices, and S is a diagonal matrix containing singular values.
- **Principal Component Analysis (PCA)**: A linear dimensionality reduction method that finds the directions (principal components) that maximize the variance of the data. The eigenvectors of the covariance matrix correspond to the principal components.
- **Matrix Factorization for Recommender Systems**: Approximates a user-item rating matrix as a product of two lower-dimensional matrices representing user and item latent factors. $r_{ij} \approx u_i \cdot v_j$, where r_{ij} is the rating of user i for item j, u_i is the user's latent factor vector, and v_j is the item's latent factor vector.
- **Modeling Systematic Biases in Recommender Systems**: $r_{ij} \approx \mu + b_i + b_j + u_i \cdot v_j$, where r_{ij} is the rating, μ is the overall mean rating, b_i is the user bias, b_j is the item bias, and $u_i \cdot v_j$ represents user-item interaction.
- **K-means Clustering**: The algorithm iteratively assigns data points to the nearest cluster centroid and then recalculates the centroids until convergence.
- **Euclidean Distance**: $d_E(x, y) = \sqrt{\sum_{i=1}^{p} (x_i - y_i)^2}$, where p is the number of attributes.
- **Hamming Distance**: Counts the number of differing bits between two binary vectors.
- **Matching Coefficient**: Hamming distance normalized by the number of attributes/bits.
- **Purity**: $purity(C, G) = \frac{1}{N} \sum_{i=1}^{K} \max_j N_j \in G_i$, where N is the total number of data points, K is the number of clusters, and N_j is the number of examples of class j in cluster i.
- **Conversion to CNF**: This algorithm transforms a logical expression into Conjunctive Normal Form (CNF) by applying logical equivalences to eliminate implications, biconditionals, and negations, and then using distributivity to flatten the expression into a conjunction of clauses.
- **SAT Problem**: This problem determines if a given propositional formula (often in CNF) is satisfiable, meaning there exists an assignment of truth values to its variables that makes the formula true.
- **3-SAT Reduction**: This algorithm reduces a general SAT problem to a 3-SAT problem by transforming clauses with more than three literals into a conjunction of clauses with exactly three literals using auxiliary variables. The resulting formula is equisatisfiable with the original.
- **Recursion**: A method that calls itself, eventually terminating in a base case. The recursive case must be "smaller" in some sense, closer to the base case.
- **Recursive Problem Solving**: Do part of the work and reduce the rest into a similar but smaller problem.
- **Structural Recursion**: Traversal of recursive data structures like trees using Breadth-First Search or Depth-First Search.
- **Structural Recursion (Binary Tree Example)**: Searching a binary search tree recursively. If the current node's value (V) equals the target value (X), return True. If there are no children, return False. If $X < V$, search the left subtree; if $X > V$, search the right subtree.

- **Revisiting Fibonacci (Optimization)**: Avoid redundant calculations by storing and reusing previously computed values (dynamic programming or memoization).
- **Naïve\textbackslash{}\textbackslash{}_SAT**: A recursive algorithm that explores all possible variable assignments to determine the satisfiability of a propositional formula. It checks for immediate satisfiability or unsatisfiability, then recursively explores assignments for an unassigned variable, backtracking if necessary.
- **DPLL**: A recursive algorithm that improves upon the Naïve\\$_S$ $ATalgorithmbyincorporatingBooleanConstraintPropaga$ $Asub-algorithmwithinDPLLthatrepeatedlyidentifiesandassignsvaluestoliteralsinunitclauses(clauseswithonlyoneun$

- **Resolution Rule**: An inference rule that derives a new clause from two existing clauses containing a literal and its negation. The new clause combines the remaining literals from the original clauses using logical OR.
- **Clause Learning**: A technique used to learn from conflicts encountered during the DPLL search. When a conflict occurs, a clause is learned that prevents the algorithm from repeating the same mistake in the future.
  - **Fairness through Blindness**: Ignore protected attributes during model training and prediction.
  - **Group Fairness**: $P(\text{outcome } o|S) = P(\text{outcome } o|T)$, where S and T are two groups.
  - **Individual Fairness**: Treat similar individuals similarly; similar individuals should have similar outcomes.
  - **Removing Bias from Models**: Keep bias features in the data during training, then remove what was learned from these features after training.